

Spring Security JWT Authentication

From Stateful Sessions to Stateless JWT Authentication

1. The Problem: Authentication on Every Request

Consider a protected endpoint:

```
@GetMapping("/profile")
public String profile(Authentication authentication) {
    return "Logged in as: " + authentication.getName();
}
```

If the request contains valid credentials:

```
GET /profile
Authorization: Basic ...
```

authentication succeeds.

But a second request without credentials is a completely separate HTTP request:

```
GET /profile
```

Result:

```
401 Unauthorized
```

Why?

HTTP does not automatically remember that a previous request was authenticated.

Conceptually:

```
Request 1
GET /profile
Authorization: Basic ...
    ↓
Authentication succeeds

Request 2
GET /profile
    ↓
No credentials available
```

Authenticating one request does not automatically authenticate future requests.

The server therefore needs some identifier travelling with later requests if it wants to remember an authenticated client.

2. Session-Based Authentication

After a successful username/password login, Spring Security has an authenticated `Authentication` object.

Conceptually:

```
Authentication
├─ Principal
├─ Authorities
└─ authenticated = true
```

Instead of discarding this authentication state after the request, the server can associate it with an HTTP session.

```
Server

Session ABC123
    ↓
SecurityContext
```

↓
Authentication
↓
aditya
ROLE_USER

The client receives only the session identifier:

ABC123

In a Spring Boot servlet application, the session ID commonly travels through a cookie:

Cookie: JSESSIONID=ABC123

Session Login Flow



Future request:

GET /profile
Cookie: JSESSIONID=ABC123

```
Find session ABC123
↓
SecurityContext
↓
Authentication
↓
User authenticated
```

`JSESSIONID` does not normally contain the username, password essentially an identifier that points to server-side session state.

3. Session Management in Spring Security

A typical session-based configuration can look like:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http
        .csrf(csrf -> csrf.disable())

        .authenticationProvider(authenticationProvider)

        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/register").permitAll()
            .anyRequest().authenticated()
        )

        .formLogin(Customizer.withDefaults())

        .httpBasic(Customizer.withDefaults())

        .sessionManagement(session ->
            session.sessionCreationPolicy(
```

```

        SessionCreationPolicy.IF_REQUIRED
    );
    );

    return http.build();
}

```

Spring Security can persist the `SecurityContext` using an `HttpSession`-backed `SecurityContextRepository`.

SessionCreationPolicy

`SessionCreationPolicy` controls how Spring Security behaves with HTTP sessions.

Policy	Creates Session?	Uses Existing Session?	Mental Model
<code>ALWAYS</code>	Yes	Yes	Always session-based
<code>IF_REQUIRED</code>	When needed	Yes	Create a session when security requires one
<code>NEVER</code>	No	Yes	Do not create a session, but use one if it already exists
<code>STATELESS</code>	No	No for restoring the <code>SecurityContext</code>	Authenticate each request independently

For session-based authentication:

```
SessionCreationPolicy.IF_REQUIRED
```

is a common choice.

For JWT authentication later:

```
SessionCreationPolicy.STATELESS
```

will be used.

4. Why Session Authentication I

With a session, the request may contain only:

```
Cookie: JSESSIONID=ABC123
```

The server still has to remember what that session represent

```
SERVER SESSION STORE
```

```
ABC123 → Aditya
```

```
XYZ456 → Rohit
```

```
PQR789 → Priya
```

Authentication state therefore exists on the server.

This is why session-based authentication is called:

| Stateful Authentication

5. Why JWT?

JWT stands for:

| JSON Web Token

The central idea is:

| Can a user authenticate once without requiring the server to maintain a separate session entry for every authenticated client?

With a session:

```
JSESSIONID = ABC123
```

```
↓
```

```
Session Store
```

```
↓
```

```
SecurityContext
```

```
↓
```

```
Authentication
  ↓
Aditya
```

The value `ABC123` tells us almost nothing by itself.

A JWT can instead carry claims such as:

```
{
  "sub": "aditya",
  "roles": ["ROLE_USER"],
  "iat": 1786570000,
  "exp": 1786573600
}
```

The server can use information inside the token to reconstruct authentication for each request.

```
JWT
  ↓
claims
  ↓
Authentication
  ↓
SecurityContext
```

The server no longer needs to preserve the user's `Authentication` object in an HTTP session.

6. The Security Problem With Client-Carried Data

Suppose the token contains:

```
{
  "sub": "aditya",
```

```
"roles": [ "ROLE_USER" ]
}
```

What prevents a client from changing it to:

```
{
  "sub": "aditya",
  "roles": [ "ROLE_ADMIN" ]
}
```

The server needs to answer two questions:

- Was this information issued by a trusted party?
- Has the information been modified after it was issued?

That is the purpose of the **signature**.

7. Why Not Send Plain JSON?

A naive token could be:

```
{
  "sub": "aditya",
  "role": "ROLE_USER"
}
```

But raw JSON is inconvenient as a transport format:

- It contains spaces, braces, quotes, and possibly line breaks.
- The same JSON can be formatted differently.
- Tokens should be easy to transport as a compact string.
- Cryptographic signing needs a precise byte representation.

JWT therefore uses **Base64URL encoding** for its compact representation.

8. Base64URL Is Encoding, Not

For example:

```
{"sub": "42"}
```

can be represented as something like:

```
eyJzdWIiOiI0MiJ9
```

Base64URL provides a URL-safe textual representation of bytes and generally removes trailing padding.

The important point is:

Base64URL is encoding, not encryption.

Anyone who gets the token can decode the header and payload.

No password or secret key is required merely to decode them.

Therefore, JWT payloads should not be treated as hidden data.

Base64URL also does not compress the payload; encoded data is generally larger than the original bytes.

9. Why Encoding Alone Is Insecure

Suppose the token is only:

```
Base64URL(payload)
```

Payload:

```
{  
  "sub": "user-42",
```

```
"role": "STUDENT"
}
```

An attacker could:

1. Decode the payload.
2. Change `STUDENT` to `ADMIN`.
3. Encode it again.
4. Send the modified value.

The server would have no cryptographic way to know whether the data was genuinely issued by it.

We need:

Integrity

The protected data has not been modified.

Authenticity

The protected data was created by someone possessing the trusted signing key.

10. Why a Plain Hash Is Not Enough

A naive idea might be:

```
token = payload + SHA256(payload)
```

or:

```
Base64URL(payload).SHA256(payload)
```

But an attacker can modify the payload and calculate a new SHA-256 hash themselves.

```
Original:
role = STUDENT

Attacker changes:
role = ADMIN

Attacker calculates:
SHA256(modified payload)
```

A plain hash can answer:

```
Does this hash match this data?
```

But it cannot answer:

```
Was this data produced by my trusted server?
```

To establish authenticity, the calculation must depend on a secret or private key that the attacker does not possess.

11. Structure of a Signed JWT

A signed JWT commonly has three parts:

```
HEADER.PAYLOAD.SIGNATURE
```

Example shape:

```
xxxxx.yyyyy.zzzzz
```

The header and payload are represented using Base64URL, and the signature bytes are also represented in Base64URL form.

11.1 Header

Example:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

alg

Specifies the cryptographic algorithm used to sign the token.

typ

Describes the token type.

11.2 Payload

Example:

```
{
  "iss": "https://auth._____.in",
  "sub": "user-42",
  "aud": "course-api",
  "role": "STUDENT",
  "iat": 1786570000,
  "exp": 1893456000
}
```

Common JWT Claims

Claim	Meaning
iss	Issuer — who issued the token
sub	Subject — who or what the token represents
aud	Audience — which application/API should accept it
iat	Issued At — when the token was created
exp	Expiration Time — when the token expires

Claim	Meaning
<code>jti</code>	Unique identifier for the token

The JWT standard defines registered claims, but not every application. The application or security protocol decided required.

Custom Claims

Applications can also add their own claims:

```
{
  "roles": ["ROLE_USER"],
  "department": "engineering"
}
```

Because the payload is readable, avoid storing unnecessary or sensitive information such as:

```
Passwords
Secret API keys
Credit card numbers
Private personal information
```

11.3 Signature

The signature protects the encoded header and encoded payload.

```
Base64URL(header) + "." + Base64URL(payload)
      ↓
    signing input
```

Conceptually:

```
signature = cryptographic_sign(
    signingInput,
```

```
    signingKey  
)
```

Final JWT:

```
Base64URL(header)  
.  
Base64URL(payload)  
.  
Base64URL(signature)
```

The signature covers both the header and the payload.

12. Symmetric JWT Signing With HS256

HS256 means:

```
HMAC using SHA-256
```

Conceptually:

```
signature =  
    HMAC-SHA256(  
        secretKey,  
        Base64URL(header) + "." + Base64URL(payload)  
    )
```

The flow is:

```
headerEncoded.payloadEncoded  
    +  
    secret key  
    ↓  
    HMAC-SHA256  
    ↓  
    signature
```

The client receives:

```
headerEncoded.payloadEncoded.signature
```

13. What Happens If the Payload

Suppose the original payload contains:

```
"role": "STUDENT"
```

An attacker changes it to:

```
"role": "ADMIN"
```

The payload encoding changes, which means the signing input changes.

```
Original:  
header.originalPayload  
  
Modified:  
header.modifiedPayload
```

Without the secret key, the attacker cannot calculate the correct HMAC for the modified input.

When the modified token reaches the server:

```
Server recalculates expected signature  
using its secret  
↓  
Expected signature ≠ token signature  
↓  
Reject token
```

The JWT signature does **not physically prevent** someone from editing the token.

It makes unauthorized modification detectable.

14. Symmetric Keys

With HS256:

```
Signing key = Verification key
```

The same secret is used to create and verify the HMAC.

```
Issuer:  
HMAC(message, secret)  
  
Verifier:  
HMAC(message, same secret)
```

Advantage

- Simple
- Fast
- Convenient when one trusted application both issues and validates tokens

Limitation

Every service that verifies the token must know the same secret.

```
Authentication server ┌  
Course service       │ same secret  
Payment service      │  
Admin service        └
```

Anyone who has the shared secret can not only verify tokens but can also create valid tokens.

If one service is compromised, the shared signing secret may also be compromised.

15. Asymmetric JWT Signing

Asymmetric cryptography uses a key pair:

```
Private key  
Public key
```

The keys have different responsibilities:

```
Private key → creates signatures  
Public key  → verifies signatures
```

Common JWT signature algorithms include:

```
RS256 → RSA signature with SHA-256  
ES256 → ECDSA signature with SHA-256
```

For RS256, conceptually:

```
signature = Sign(  
    privateKey,  
    Base64URL(header) + "." + Base64URL(payload)  
)
```

Verification:

```
Verify(  
    publicKey,  
    signingInput,  
    signature  
)
```

A typical architecture becomes:

```
Authentication server → private key + public key  
Course service        → public key only
```

```
Payment service    → public key only
Admin service      → public key only
```

The resource services can verify signatures using the public
create new valid signatures.

This model is useful when one authorization server issues to
APIs validate them.

16. Symmetric vs Asymmetric Signing

Property	HS256	RS256 / ES256
Key model	Shared secret	Private/public key pair
Signing	Secret key	Private key
Verification	Same secret key	Public key
Can verifier create tokens?	Yes, if it has the secret	No, if it only has the public key
Simple for one application	Yes	More setup
Useful for multiple independent APIs	Less ideal	Very useful

17. JWT Tampering Experiment

A useful way to understand JWT security is to create a token, decode it, modify its payload, and then verify the modified token.

Original Token Shape

```
A.B.C
```

Where:

A = encoded header
B = encoded payload
C = signature

Suppose the original payload contains:

```
{  
  "sub": "aditya",  
  "role": "STUDENT"  
}
```

After changing the role to `ADMIN`, the new encoded payload becomes:

B2

If a tool has access to the secret, it can generate a valid new signature:

A.B2.C2

But a real attacker who does not know the secret cannot calculate `C2`.

The attacker's tampered token would instead look like:

A.B2.C

That is:

```
Original header  
+  
Modified payload  
+  
Original signature
```

The payload can still be decoded and may visibly show:

```
{  
  "sub": "aditya",  
  "role": "ADMIN"  
}
```

But signature verification fails because the signature was created with the original payload, not the modified payload.

```
Modified signing input  
    ↓  
Old signature no longer matches  
    ↓  
Invalid Signature
```

This demonstrates the central security property of a signed JWT:

The payload is readable and editable, but unauthorized changes cause signature verification to fail.

18. JWT Authentication Flow in the Application

There are two major flows.

Login Flow

```
POST /auth/login  
  
{  
  "username": "aditya",  
  "password": "123456"  
}  
  
    ↓  
AuthenticationManager  
    ↓  
DaoAuthenticationProvider  
    ↓  
CustomUserDetailsService
```

```
↓  
Database  
↓  
PasswordEncoder  
↓  
Authentication successful  
↓  
Generate JWT  
↓  
Return token
```

Future Protected Request

```
GET /profile  
  
Authorization:  
Bearer eyJhbGciOi...  
↓  
Extract JWT  
↓  
Validate JWT  
↓  
Identify user  
↓  
Create Authentication  
↓  
SecurityContext  
↓  
Authorization  
↓  
Controller
```

19. Add Spring Security Resource Server Support

Dependency:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-oauth2-resource-server</a
```

```
artifactId>  
</dependency>
```

Spring Security Resource Server can authenticate requests u
including custom JWTs issued by the application itself.

20. **JwtEncoder** vs **JwtDecoder**

Spring Security separates token creation and token validation into two responsibilities.

```
JwtEncoder  
  ↓  
CREATE / SIGN JWT
```

```
JwtDecoder  
  ↓  
READ / VERIFY / VALIDATE JWT
```

For this application, HS256 is used:

```
same secret key  
  ↓  
sign JWT  
  ↓  
verify JWT
```

This keeps the demonstration simple because the same Spring Boot application
both issues and validates the token.

21. Configure the JWT Secret

Generate a sufficiently long Base64-encoded secret:

```
openssl rand -base64 32
```

Configuration:

```
jwt.secret=${JWT_SECRET}
jwt.issuer= -api
jwt.expiry=900
```

Here:

```
900 seconds = 15 minutes
```

For classroom demonstrations, the secret can temporarily be placed directly in `application.properties`.

For a real application:

| Do not commit signing secrets to Git.

22. Create the `SecretKey`

```
@Bean
public SecretKey jwtSecretKey(
    @Value("${jwt.secret}") String secret) {

    byte[] decodedKey =
        Base64.getDecoder().decode(secret);

    return new SecretKeySpec(
        decodedKey,
        "HmacSHA256"
    );
}
```

Conceptually:

```
Base64 configuration value
    ↓
decode
    ↓
raw key bytes
    ↓
SecretKey
```

The same `SecretKey` will be used by both the encoder and decoder.

23. Create the `JwtEncoder`

```
@Bean
public JwtEncoder jwtEncoder(
    SecretKey secretKey) {

    return NimbusJwtEncoder
        .withSecretKey(secretKey)
        .algorithm(MacAlgorithm.HS256)
        .build();
}
```

Application dependency:

```
Application
    ↓
JwtEncoder
    ↓
NimbusJwtEncoder
```

The rest of the application depends on the `JwtEncoder` abstraction, while `NimbusJwtEncoder` provides the implementation.

24. Login DTOs

Login request:

```
public class LoginRequest {  
    String username;  
    String password;  
}
```

Login response:

```
public record LoginResponse(  
    String accessToken  
) {}
```

Client request:

```
{  
  "username": "aditya",  
  "password": "123456"  
}
```

Server response:

```
{  
  "accessToken": "eyJhbGciOiJIUzI1NiJ9..."  
}
```

25. Create **JwtService**

```
@Service  
public class JwtService {  
  
    private final JwtEncoder jwtEncoder;
```

```

@Value("${jwt.issuer}")
private String issuer;

@Value("${jwt.expiry}")
private long expiry;

public JwtService(JwtEncoder jwtEncoder) {
    this.jwtEncoder = jwtEncoder;
}

public String generateToken(
    Authentication authentication) {

    Instant now = Instant.now();

    List<String> authorities =
        authentication.getAuthorities()
            .stream()
            .map(GrantedAuthority::getAuthority)
            .toList();

    JwtClaimsSet claims =
        JwtClaimsSet.builder()

            .issuer(issuer)

            .issuedAt(now)

            .expiresAt(
                now.plusSeconds(expiry)
            )

            .subject(
                authentication.getName()
            )

```

```

        .claim(
            "authorities",
            authorities
        )

        .build();

    Jwt jwt =
        jwtEncoder.encode(
            JwtEncoderParameters.from(claims)
        );

    return jwt.getTokenValue();
}
}

```

26. Understanding **JwtService**

The service receives an already authenticated:

Authentication authentication

Conceptually:

```

Principal    → aditya
Authorities  → ROLE_USER
Authenticated → true

```

The service converts this successful authentication result into JWT claims.

issuer

```
.issuer(issuer)
```

Produces:

```
"iss": "[redacted]-api"
```

It identifies who issued the JWT.

The decoder can later be configured to accept tokens only from this issuer.

subject

```
.subject(authentication.getName())
```

Produces:

```
"sub": "aditya"
```

The subject identifies who the token represents.

issuedAt

```
.issuedAt(now)
```

Produces the:

```
iat
```

claim.

It records when the token was created.

expiresAt

```
.expiresAt(  
    now.plusSeconds(expiry)  
)
```

Produces the:

```
exp
```

claim.

After this time, the token should no longer be accepted.

27. Why Store Authorities in the JWT?

The application adds:

```
.claim(  
    "authorities",  
    authorities  
)
```

Example payload:

```
{  
  "authorities": [  
    "ROLE_USER"  
  ]  
}
```

The JWT then contains enough information to recreate the user's authentication.

```
sub  
↓  
Who is the user?  
  
authorities
```

↓
What authorities does the user have?

Subsequent requests do not need this flow:

JWT
↓
username
↓
database
↓
UserDetails

Instead:

JWT
├─ sub = aditya
└─ authorities = ROLE_USER
↓
Authentication

This is what makes the example stateless.

28. Create `/auth/login`

```
@RestController
@RequestMapping("/auth")
public class AuthController {

    private final AuthenticationManager
        authenticationManager;

    private final JwtService jwtService;

    public AuthController(
        AuthenticationManager authenticationManager,
        JwtService jwtService) {
```

```

        this.authenticationManager =
            authenticationManager;

        this.jwtService = jwtService;
    }

    @PostMapping("/login")
    public LoginResponse login(
        @RequestBody LoginRequest request) {

        Authentication authenticationRequest =
            UsernamePasswordAuthenticationToken
                .unauthenticated(
                    request.username(),
                    request.password()
                );

        Authentication authentication =
            authenticationManager.authenticate(
                authenticationRequest
            );

        String token =
            jwtService.generateToken(
                authentication
            );

        return new LoginResponse(token);
    }
}

```

Login Authentication Flow

```
username + password
    ↓
UsernamePasswordAuthenticationToken
unauthenticated
    ↓
AuthenticationManager
    ↓
DaoAuthenticationProvider
    ↓
CustomUserDetailsService
    ↓
Database
    ↓
PasswordEncoder
    ↓
Authentication
authenticated = true
    ↓
JwtService
    ↓
JwtEncoder
    ↓
JWT
```

The username/password authentication mechanism remains the same as before. JWT generation happens **after** authentication succeeds.

29. Authenticate Future Requests Using JWT

JWT creation is only half of the process.

When a token comes back from the client, the application must:

```
JWT string
    ↓
verify signature
    ↓
validate claims
    ↓
trusted Jwt object
```

This is the responsibility of `JwtDecoder`.

30. Create `JwtDecoder`

```
@Bean
public JwtDecoder jwtDecoder(
    SecretKey secretKey,
    @Value("${jwt.issuer}") String issuer) {

    NimbusJwtDecoder decoder =
        NimbusJwtDecoder
            .withSecretKey(secretKey)
            .macAlgorithm(MacAlgorithm.HS256)
            .build();

    decoder.setJwtValidator(
        JwtValidators.createDefaultWithIssuer(
            issuer
        )
    );

    return decoder;
}
```

The decoder uses the same HS256 secret key that was used by the encoder. It also configures validation for the expected issuer.

31. What the Decoder Checks

For an incoming token:

```
HEADER.PAYLOAD.SIGNATURE
```

the decoder performs security checks conceptually like:

```
Is this structurally a valid JWT?  
  ↓  
Can the signature be verified  
using the trusted secret key?  
  ↓  
Are the standard claims valid?  
  ↓  
Is the issuer correct?  
  ↓  
Is the token currently valid?
```

Only after these checks can the resulting claims be treated as trusted authentication data.

32. Map JWT Authorities Into Spring Security

Spring Security Resource Server commonly maps OAuth scopes such as:

```
{  
  "scope": "read write"  
}
```

into authorities like:

```
SCOPE_read  
SCOPE_write
```

But this application stores:

```
ROLE_USER  
ROLE_ADMIN
```

inside a custom JWT claim:

```
{  
  "authorities": [  
    "ROLE_USER"  
  ]  
}
```

Therefore, Spring must be told:

1. Read authorities from the `authorities` claim.
 2. Do not add another prefix.
-

33. `JwtAuthenticationConverter`

```
@Bean  
public JwtAuthenticationConverter  
    jwtAuthenticationConverter() {  
  
    JwtGrantedAuthoritiesConverter  
        authoritiesConverter =  
            new JwtGrantedAuthoritiesConverter();  
  
    authoritiesConverter  
        .setAuthoritiesClaimName(  
            "authorities"  
        );  
  
    authoritiesConverter  
        .setAuthorityPrefix("");  
  
    JwtAuthenticationConverter  
        authenticationConverter =  
            new JwtAuthenticationConverter();  
  
    authenticationConverter
```

```

        .setJwtGrantedAuthoritiesConverter(
            authoritiesConverter
        );

    return authenticationConverter;
}

```

The empty prefix is important because the token already contains:

```
ROLE_USER
```

The application wants exactly:

```
ROLE_USER
```

not an automatically prefixed value.

This keeps existing authorization rules compatible:

```
.hasRole("USER")
```

```
.hasRole("ADMIN")
```

34. Final **SecurityFilterChain**

```

@Bean
public SecurityFilterChain securityFilterChain(
    HttpSecurity http,
    JwtAuthenticationConverter
        jwtAuthenticationConverter)
    throws Exception {

    http

```

```

        .csrf(csrf ->
            csrf.disable()
        )

        .sessionManagement(session ->
            session.sessionCreationPolicy(
                SessionCreationPolicy.STATELESS
            )
        )

        .authorizeHttpRequests(auth ->
            auth

                .requestMatchers(
                    "/auth/login",
                    "/register"
                )
                .permitAll()

                .requestMatchers(
                    "/admin/**"
                )
                .hasRole("ADMIN")

                .anyRequest()
                .authenticated()
            )

        .oauth2ResourceServer(oauth2 ->
            oauth2.jwt(jwt ->
                jwt.jwtAuthenticationConverter(
                    jwtAuthenticationConverter
                )
            )
        )
    );

```

```
    return http.build();  
}
```

35. Why

SessionCreationPolicy.STATELESS ?

```
.sessionManagement(session ->  
    session.sessionCreationPolicy(  
        SessionCreationPolicy.STATELESS  
    )  
)
```

means Spring Security should not use the HTTP session to preserve authentication between requests.

Each protected request must carry its own authentication material, in this case:

```
Authorization: Bearer <JWT>
```

The server validates the token and reconstructs authentication for that request.

```
Request  
+  
Bearer JWT  
↓  
JWT validation  
↓  
Authentication  
↓  
SecurityContext  
↓  
Authorization
```

After the request completes, the application does not rely on an HTTP session to remember that authentication for the next request.

36. Why `oauth2ResourceServer()`

```
.oauth2ResourceServer(oauth2 ->
    oauth2.jwt(jwt ->
        jwt.jwtAuthenticationConverter(
            jwtAuthenticationConverter
        )
    )
)
```

Spring Security packages bearer-token JWT authentication under its **Resource Server** support.

This support can protect an API using a custom JWT even when the application is not implementing a complete OAuth2 Authorization Server.

Conceptually:

```
graph TD
    A[Protected request] --> B[Authorization: Bearer <JWT>]
    B --> C[Spring Security Resource Server]
    C --> D[JWT validation]
    D --> E[Authentication]
```

37. Two Authentication Providers, Two Purposes

The application now uses two different authentication mechanisms.

Username/Password Login

```
DaoAuthenticationProvider
```

```
understands:  
username + password
```

It authenticates the user's login credentials.

JWT Authentication

```
JwtAuthenticationProvider  
_____  
understands:  
JWT
```

It authenticates future requests carrying bearer tokens.

Both mechanisms eventually produce the same core Spring Security abstraction:

```
Authentication
```

This keeps Spring Security's architecture consistent regardless of how the user proves their identity.

38. Complete End-to-End Flow

First Request: Login

```
POST /auth/login  
username + password  
  ↓  
AuthenticationManager  
  ↓  
DaoAuthenticationProvider  
  ↓  
CustomUserDetailsService  
  ↓  
PasswordEncoder  
  ↓  
Authentication  
  ↓  
JwtService
```

```
↓  
JwtEncoder  
↓  
Signed JWT  
↓  
Return token to client
```

Future Request

```
GET /profile  
Authorization: Bearer <JWT>  
↓  
Bearer-token JWT authentication  
↓  
JwtDecoder  
↓  
Verify signature  
↓  
Validate claims  
↓  
JwtAuthenticationConverter  
↓  
Create Authentication  
↓  
SecurityContext  
↓  
Authorization rules  
↓  
Controller
```

39. Session Authentication vs JWT Authentication

Session-Based Authentication	JWT-Based Authentication
Server stores authentication state	Authentication information is reconstructed from the token
Client usually sends <code>JSESSIONID</code>	Client sends a bearer JWT
Session ID points to server-side state	Token carries claims
Commonly stateful	Can be stateless

Session-Based Authentication	JWT-Based Authentication
Server restores <code>SecurityContext</code> from session	Server validates token and per request

40. Important JWT Takeaways

- JWT stands for **JSON Web Token**.
- A signed JWT commonly has three parts:

HEADER . PAYLOAD . SIGNATURE

- Header and payload are encoded, not encrypted.
- Anyone possessing a normal signed JWT can usually read its claims.
- The signature provides integrity and authenticity.
- The signature does not stop a person from editing the token; it makes unauthorized changes detectable.
- HS256 uses the same secret for signing and verification.
- Asymmetric signing uses a private key for signing and a public key for verification.
- Do not place passwords or sensitive secrets inside JWT payloads.
- `JwtEncoder` creates/signs JWTs.
- `JwtDecoder` verifies and validates incoming JWTs.
- JWT claims can be used to reconstruct Spring Security's `Authentication`.
- `SessionCreationPolicy.STATELESS` prevents Spring Security from relying on an HTTP session to preserve authentication between requests.
- Spring Security Resource Server support can validate custom bearer JWTs.
- `DaoAuthenticationProvider` handles username/password authentication.
- `JwtAuthenticationProvider` handles JWT authentication.

- Both ultimately produce an `Authentication` object.

41. Mental Model

LOGIN

```
Username + Password
  ↓
AuthenticationManager
  ↓
DaoAuthenticationProvider
  ↓
UserDetailsService + PasswordEncoder
  ↓
Authentication
  ↓
JwtEncoder
  ↓
Signed JWT
  ↓
Client
```

PROTECTED REQUEST

```
Client
  ↓
Authorization: Bearer <JWT>
  ↓
JWT verification + validation
  ↓
JwtAuthenticationProvider
  ↓
JwtAuthenticationConverter
  ↓
Authentication
  ↓
SecurityContext
  ↓
Authorization
  ↓
Controller
```

With session authentication, the server remembers the authentication state between requests. With stateless JWT authentication, each request contains the information needed to recreate and verify authentication.